TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO PHẦN 3: LỚP MẢNG ĐỐI TƯỢNG, FILE TEXT & STATIC

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (JAVA OOP) • PHẦN 3

Phân công: Thành viên 3 | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

HƯỚNG DẪN HỌC & BẢO VỆ VẤN ĐÁP ĐỒ ÁN OOP (ĐIỂM 10/10)

> **Phân công thành viên:** Thành viên 3

> **Các lớp phụ trách (4 lớp & 3 file dữ liệu):** `DanhSachSanPham` , `DanhSachKhachHang` , `ChiTietHoaDon` , `HoaDon` , `products.txt` , `users.txt` , `hoadon.txt` .

> **Nhiệm vụ cốt lõi:** Nắm vững cấu trúc Lớp mảng danh sách đối tượng (Class Array), cơ chế tự sinh mã duy nhất, kỹ thuật Đọc/Ghi file text chuẩn UTF-8 chống lỗi font tiếng Việt và ý nghĩa các thành phần Static.

1. MÔ HÌNH LỚP MẢNG & DÒNG CHẢY DỮ LIỆU FILE

```
[products.txt]  <--->  [DanhSachSanPham]  <--->  [QuanLyCuaHang]
                                |                       |
                           (quản lý)               (quản lý)
                                v                       v
                      ArrayList<SanPhamThoiTrang>     [HoaDon]
                                                        | (chứa)
   [users.txt]     <--->  [DanhSachKhachHang]           v
                                |                  ArrayList<ChiTietHoaDon>
                           (quản lý)                    |
                                v                  (lưu vết)
                          ArrayList<Nguoi>              v
                                                   [hoadon.txt]
```

- **Lớp mảng (Class Array):** Đóng gói bộ sưu tập `ArrayList` bên trong class, không để lộ cấu trúc dữ liệu ra ngoài, cung cấp các hàm nghiệp vụ chuẩn: Thêm, Xóa, Sửa, Tìm kiếm, Tự sinh mã.
- **Lưu trữ dữ liệu:** Đọc/Ghi đồng bộ dữ liệu kho hàng và tài khoản bằng luồng `UTF-8` .

2. GIẢI THÍCH CHI TIẾT TỪNG ĐOẠN CODE CỐT LÕI

Đoạn 1: Thuật toán Tự sinh mã ID tự động (`taoMaTuDong`)

```
public String taoMaTuDong(String type) {
        String prefix = "SP";
        if (type != null) {
            String t = type.toLowerCase();
            if (t.equals("aokhoac")) prefix = "AK";
            else if (t.equals("quanjean") || t.equals("jeans")) prefix = "QJ";
            else if (t.equals("aothun") || t.equals("tshirt")) prefix = "AT";
            else if (t.equals("phukien") || t.equals("accessory")) prefix = "PK";
            else if (t.equals("vaydam")) prefix = "VD";
            else if (t.equals("giaydep")) prefix = "GD";
        }
```

```
        int maxId = 0;
        for (SanPhamThoiTrang p : danhSach) {
            if (p.getMaSP() != null && p.getMaSP().toUpperCase().startsWith(prefix)) {
                String numPart = p.getMaSP().substring(prefix.length());
                try {
                    int id = Integer.parseInt(numPart);
                    if (id > maxId) maxId = id;
                } catch (NumberFormatException ignored) {}
            }
        }
        return String.format("%s%03d", prefix, maxId + 1);
    }
```

- **Giải thích từng dòng code:**
- `String prefix` : Xác định tiền tố theo loại sản phẩm (`AK` cho Áo khoác, `QJ` cho Quần jean, `GD` cho Giày dép...).
- Vòng lặp `for (SanPhamThoiTrang p : danhSach)` : Quét toàn bộ danh sách sản phẩm hiện có trong kho.
- `p.getMaSP().toUpperCase().startsWith(prefix)` : Lọc những sản phẩm có cùng tiền tố.
- `p.getMaSP().substring(prefix.length())` : Cắt bỏ phần chữ cái để lấy phần số thứ tự phía sau. Ví dụ `"AK005".substring(2)` lấy được `"005"` .
- `Integer.parseInt(numPart)` : Chuyển chuỗi số thành số nguyên và tìm giá trị lớn nhất `maxId` .
- `String.format("%s%03d", prefix, maxId + 1)` : Sinh mã mới bằng tiền tố kết hợp số tiếp theo được đệm 3 chữ số (VD: `AK001` , `AK002` ...). Người dùng không cần tự gõ mã bằng tay, tránh 100% nguy cơ trùng lặp mã sản phẩm.

Đoạn 2: Thao tác Thêm, Sửa, Xóa trong Lớp mảng `DanhSachSanPham`

```
public boolean them(SanPhamThoiTrang sp) {
        if (sp == null || sp.getMaSP() == null || sp.getMaSP().trim().isEmpty()) {
            return false;
        }
        if (timKiem(sp.getMaSP()) != null) {
            System.out.println("San pham co ma " + sp.getMaSP() + " da ton tai!");
            return false;
        }
        danhSach.add(sp);
        return true;
    }

    public boolean xoa(String maSP) {
        if (maSP == null) return false;
        for (int i = 0; i < danhSach.size(); i++) {
            if (danhSach.get(i).getMaSP() != null &&
danhSach.get(i).getMaSP().equalsIgnoreCase(maSP.trim())) {
                danhSach.remove(i);
                return true;
            }
        }
        return false;
    }

    public boolean sua(String maSP, String tenMoi, double giaMoi) {
        SanPhamThoiTrang sp = timKiem(maSP);
        if (sp == null) return false;
```

```
        if (tenMoi != null && !tenMoi.trim().isEmpty()) sp.setTenSP(tenMoi.trim());
        if (giaMoi >= 0) sp.setGiaBan(giaMoi);
        return true;
    }
```

- **Giải thích từng dòng code:**
- `them(...)` : Kiểm tra mã sản phẩm đã tồn tại hay chưa bằng hàm `timKiem` . Nếu đã có thì từ chối thêm (tránh trùng khóa chính). Trả về `boolean` thể hiện trạng thái.
- `xoa(...)` : Dùng vòng lặp tìm kiếm đối tượng có mã khớp không phân biệt hoa thường (`equalsIgnoreCase`). Khi tìm thấy, gọi `danhSach.remove(i)` và dừng lặp ngay.
- `sua(...)` : Tìm đối tượng theo mã, sau đó cập nhật tên mới và giá bán mới nếu dữ liệu hợp lệ.

Đoạn 3: Kỹ thuật Đọc/Ghi File Text chuẩn UTF-8

```
public void docTuFile(String fileName) {
        File file = new File(fileName);
        if (!file.exists()) return;

        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.trim().isEmpty()) continue;
                String[] parts = line.split(",");
                if (parts.length < 5) continue;

                String type = parts[0].trim();
                String code = parts[1].trim();
                String name = parts[2].trim();
                double price = Double.parseDouble(parts[3].trim());

                switch (type.toLowerCase()) {
                    case "aokhoac":
                        boolean haveHat = Boolean.parseBoolean(parts[4].trim());
                        them(new AoKhoac("AoKhoac", code, name, price, haveHat));
                        break;
                    case "giaydep":
                        int szGiay = Integer.parseInt(parts[4].trim());
                        them(new GiayDep("GiayDep", code, name, price, szGiay));
                        break;
                    // ... các loại sản phẩm khác
                }
            }
        } catch (IOException e) {
            System.out.println("Da xay ra loi khi doc du lieu tu file.");
        }
    }
```

- **Giải thích từng dòng code:**
- `try (...)` : Cú pháp **Try-with-resources** (từ Java 7 trở lên). Đảm bảo luồng đọc file `reader` luôn được tự động đóng (`close()`) ngay cả khi có ngoại lệ xảy ra, tránh rò rỉ bộ nhớ (Memory Leak) hoặc khóa file.

- `new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8)` : Chỉ định tường minh bảng mã **UTF-8**. Giúp đọc chính xác tiếng Việt có dấu ("Áo gió ấm", "Váy dạ hội") trên mọi môi trường Windows mà không bị biến thành ký tự lạ (`??`).
- `line.split(",")` : Tách từng dòng văn bản thành mảng các chuỗi con dựa trên dấu phẩy.
- Cấu trúc `switch-case` : Phân tích cột đầu tiên (`type`) để khởi tạo chính xác đối tượng lớp con tương ứng (Kỹ thuật Factory Pattern).

Đoạn 4: Ghi dữ liệu xuống File Text

```
public void ghiVaoFile(String fileName) {
        try (BufferedWriter writer = new BufferedWriter(
                new OutputStreamWriter(new FileOutputStream(fileName), StandardCharsets.UTF_8))) {
            for (SanPhamThoiTrang sp : danhSach) {
                writer.write(sp.getStringSaveFile());
                writer.newLine();
            }
            writer.flush();
        } catch (IOException e) {
            System.out.println("Da xay ra loi khi luu du lieu vao file.");
        }
    }
```

- **Giải thích từng dòng code:**
- `BufferedWriter` và `OutputStreamWriter` kết hợp `StandardCharsets.UTF_8` : Ghi dữ liệu dạng luồng đệm tốc độ cao và chuẩn font tiếng Việt.
- `sp.getStringSaveFile()` : Gọi phương thức đa hình của từng đối tượng sản phẩm để lấy chuỗi định dạng CSV tương ứng.
- `writer.newLine()` : Xuống dòng tương thích với hệ điều hành đang chạy (CRLF trên Windows).
- `writer.flush()` : Ép toàn bộ dữ liệu còn lưu trong bộ nhớ đệm ghi ngay xuống đĩa cứng.

Đoạn 5: Chi Tiết Hóa Đơn (`ChiTietHoaDon`) & Quản lý Hóa đơn (`HoaDon`)

```
public class ChiTietHoaDon {
    private SanPhamThoiTrang sanPham;
    private int soLuong;

    public void tangSoLuong(int amount) {
        if (amount > 0) this.soLuong += amount;
    }

    public double tinhThanhTienSauGiam(double phanTramGiamGia) {
        if (sanPham == null) return 0;
        double giaSauGiam1SP = sanPham.calculateAfterDiscount(phanTramGiamGia);
        return (giaSauGiam1SP + sanPham.VATcalculation()) * soLuong;
    }
}
```

- **Giải thích từng dòng code:**
- Biểu diễn quan hệ chứa đựng (**Aggregation**): Một Hóa đơn gồm nhiều Chi tiết hóa đơn; mỗi Chi tiết hóa đơn giữ tham chiếu đến một `SanPhamThoiTrang` và số lượng mua `soLuong` .
- `tangSoLuong` : Khi khách hàng nhập trùng mã sản phẩm đã có trong giỏ hàng, hệ thống không tạo dòng mới mà cộng dồn số lượng.

- `tinhThanhTienSauGiam` : Tính tổng tiền cho dòng hàng bao gồm: `(Đơn giá sau giảm + Thuế VAT) * Số lượng` .

Đoạn 6: Các Thành phần Static (Biến Static & Hàm Static)

```
public static final double TI_LE_VAT = 0.10;
private static int tongSoLuongSanPhamDaTao = 0;
private static int tongSoHoaDon = 0;

public static String formatTien(double amount) {
    DecimalFormatSymbols symbols = new DecimalFormatSymbols();
    symbols.setGroupingSeparator('.');
    DecimalFormat df = new DecimalFormat("#,##0", symbols);
    return df.format(Math.round(amount)) + " VND";
}
```

- **Giải thích từng dòng code:**
- `public static final double TI_LE_VAT = 0.10` : Hằng số tĩnh đại diện cho mức thuế VAT 10%. Được truy xuất trực tiếp qua tên lớp: `SanPhamThoiTrang.TI_LE_VAT` mà không cần khởi tạo đối tượng.
- `tongSoLuongSanPhamDaTao` & `tongSoHoaDon` : Biến đếm mức lớp (Class-level variable). Cả chương trình chỉ tồn tại đúng 1 vùng nhớ duy nhất chia sẻ cho mọi thực thể để thống kê số lượng.
- `formatTien(double)` : Phương thức tĩnh tiện ích. Chuyển đổi số thực sang định dạng tiền tệ Việt Nam có dấu chấm phân cách hàng nghìn (ví dụ: `1800000 -> 1.800.000 VND`).

3. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU & CÁCH TRẢ LỜI ĐẠT 10 ĐIỂM

CÂU HỎI VẤN ĐÁP: Tại sao phải sử dụng `StandardCharsets.UTF_8` khi đọc và ghi file? Nếu chỉ dùng `new FileReader("products.txt")` thì có vấn đề gì?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, nếu chỉ dùng `new FileReader(file)` thì Java sẽ sử dụng bảng mã mặc định của hệ điều hành máy khách (trên Windows tiếng Việt thường là Windows-1258 hoặc Windows-1252). Khi đó, các tên sản phẩm có dấu tiếng Việt như "Áo khoác gió", "Váy dạ hội" sẽ bị biến thành các ký tự rác (lỗi font). Việc dùng `InputStreamReader` và `OutputStreamWriter` kết hợp `StandardCharsets.UTF_8` ép buộc mọi thao tác đọc/ghi phải dùng chuẩn quốc tế UTF-8, đảm bảo hiển thị hoàn hảo 100% tiếng Việt trên mọi hệ điều hành.

CÂU HỎI VẤN ĐÁP: Trong khối lệnh đọc file, cú pháp `try (BufferedReader reader = ...)` có ý nghĩa gì đặc biệt?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, đây là cấu trúc **Try-with-resources** của Java. Mọi đối tượng implements interface `AutoCloseable` được khai báo trong ngoặc đơn của `try` sẽ được hệ thống tự động đóng tài nguyên (gọi hàm `close()`) sau khi khối lệnh kết thúc, kể cả khi xảy ra Exception. Cách viết này an toàn hơn khối `try-catch-finally` truyền thống, giúp mã nguồn gọn gàng và ngăn ngừa rò rỉ tài nguyên luồng file.

CÂU HỎI VẤN ĐÁP: Thuộc tính static khác với thuộc tính thông thường (instance variable) như thế nào?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy:
• Thuộc tính thông thường thuộc về từng đối tượng cụ thể. Mỗi khi `new` một đối tượng, một vùng nhớ mới được cấp phát riêng biệt.

• Thuộc tính static thuộc về lớp. Nó chỉ được cấp phát một vùng nhớ duy nhất trong bộ nhớ Heap/Metaspace khi nạp lớp, và được chia sẻ chung cho tất cả các đối tượng của lớp đó. Trong đồ án, biến `TI_LE_VAT` và `tongSoHoaDon` là static vì chúng mang giá trị dùng chung toàn cục cho cửa hàng.